Task 2: Sparse Evolutionary Training (SET)

In the second task, students will implement Sparse Evolutionary Training (SET), which allows the sparse connectivity pattern to evolve during training. At regular intervals during training: - Remove a fraction of the smallest magnitude weights - Add the same number of new connections at random locations

This procedure maintains a constant sparsity level while enabling the network structure to adapt over time. Students should compare the performance of SET with the static sparse baseline.

0. Setup and Data Loading

```
import os
import sys
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import joblib
```

```
# If running in Colab, uncomment these lines after uploading or mounting Drive.
from google.colab import drive

drive.mount("/content/drive", force_remount=True)

PROJECT_DIR = "/content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project"
if PROJECT_DIR not in sys.path:
    sys.path.append(PROJECT_DIR)

print(os.listdir(PROJECT_DIR))
```

```
Mounted at /content/drive
['results']
```

```
results_dir = os.path.join(PROJECT_DIR, "results")

print("Results exists:", os.path.exists(results_dir))
print("Results path:", results_dir)
```

```
Results exists: True
Results path: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
use_pin_memory = torch.cuda.is_available()
num_workers = min(2, os.cpu_count() or 1)

print(f"Using device: {device}")
print(f"Project directory: {PROJECT_DIR}")
```

```
Using device: cuda
Project directory: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project
```

```
from models import ResNetCIFAR
```

```
transform = transforms.Compose(
    [
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ]
)

trainset = torchvision.datasets.CIFAR10(
    root=os.path.join(PROJECT_DIR, "data"),
    train=True,
    download=True,
    transform=transform,
)
trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=128,
    shuffle=True,
```

```
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)

testset = torchvision.datasets.CIFAR10(
    root=os.path.join(PROJECT_DIR, "data"),
    train=False,
    download=True,
    transform=transform,
)
testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=128,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=use_pin_memory,
)

classes = trainset.classes
print(f"Loaded CIFAR-10 with {len(classes)} classes.")
```

```
100%|      | 170M/170M [00:13<00:00, 12.9MB/s]

Loaded CIFAR-10 with 10 classes.
```

1. SET Configuration

```
sparsity_levels = [0.90, 0.95, 0.98]
optimizers = ["sgd", "adam"]
prune_fraction = 0.05
rewire_interval = 2
epochs = 15

criterion = nn.CrossEntropyLoss()
output_dir = os.path.join(PROJECT_DIR, "results")
os.makedirs(output_dir, exist_ok=True)

print(f"Sparsity levels: {sparsity_levels}")
print(f"Optimizers: {optimizers}")
```

```
print(f"Prune fraction: {prune_fraction:.2f}")
print(f"Rewire interval: every {rewire_interval} epoch(s)")
print(f"Output directory: {output_dir}")
```

```
Sparsity levels: [0.9, 0.95, 0.98]
Optimizers: ['sgd', 'adam']
Prune fraction: 0.30
Rewire interval: every 1 epoch(s)
Output directory: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results
```

```
def initialize_masks(model, sparsity):
    """Initialize binary masks for weight tensors based on target sparsity."""
    masks = {}
    for name, param in model.named_parameters():
        if "weight" in name and param.ndim > 1:
            mask = (torch.rand_like(param) > sparsity).float()
            masks[name] = mask
    return masks


def apply_masks(model, masks):
    """Zero out pruned weights by applying masks in-place."""
    with torch.no_grad():
        for name, param in model.named_parameters():
            if name in masks:
                param.mul_(masks[name])


def apply_masks_to_grads(model, masks):
    """Keep pruned connections frozen by masking their gradients."""
    for name, param in model.named_parameters():
        if name in masks and param.grad is not None:
            param.grad.mul_(masks[name])


def prune_and_regrow(model, masks, prune_fraction):
    """Magnitude prune active weights, then randomly regrow the same count."""
    with torch.no_grad():
        for name, param in model.named_parameters():
            if name not in masks:
                continue
```

```
            weights = param
            mask = masks[name]

            active_positions = mask.nonzero(as_tuple=False)
            if active_positions.numel() == 0:
                continue

            active_weights = weights[mask.bool()]
            n_prune = int(active_weights.numel() * prune_fraction)
            if n_prune < 1:
                continue

            _, prune_indices = torch.topk(
                active_weights.abs(), k=n_prune, largest=False
            )
            prune_positions = active_positions[prune_indices]
            prune_index = tuple(
                prune_positions[:, dim] for dim in range(prune_positions.size(1))
            )

            mask[prune_index] = 0.0
            weights[prune_index] = 0.0

            zero_positions = (mask == 0).nonzero(as_tuple=False)
            if zero_positions.numel() == 0:
                continue

            n_grow = min(n_prune, zero_positions.size(0))
            grow_choices = torch.randperm(
                zero_positions.size(0), device=zero_positions.device
            )[:n_grow]
            grow_positions = zero_positions[grow_choices]
            grow_index = tuple(
                grow_positions[:, dim] for dim in range(grow_positions.size(1))
            )

            mask[grow_index] = 1.0
            std = weights.std()
            if std == 0:
                std = 0.01
            weights[grow_index] = torch.randn(n_grow, device=weights.device) * std
```

```
def build_sparse_model(sparsity):
    model = ResNetCIFAR().to(device)
    masks = initialize_masks(model, sparsity)
    apply_masks(model, masks)
    return model, masks
```

2. Train and Save Results

```
def build_optimizer(model, optimizer_type):
    if optimizer_type == "sgd":
        return optim.SGD(model.parameters(), lr=0.05, momentum=0.9, weight_decay=5e-4)
    if optimizer_type == "adam":
        return optim.Adam(model.parameters(), lr=0.0005, weight_decay=5e-4)
    raise ValueError(f"Unsupported optimizer: {optimizer_type}")


def evaluate(model, loader, criterion):
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_examples = 0

    with torch.no_grad():
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            total_loss += loss.item()
            _, predicted = outputs.max(1)
            total_examples += labels.size(0)
            total_correct += predicted.eq(labels).sum().item()

    average_loss = total_loss / len(loader)
    accuracy = 100.0 * total_correct / total_examples
    return average_loss, accuracy


all_results = {}
```

```
for opt_name in optimizers:
    all_results[opt_name] = {}
    for target_sparsity in sparsity_levels:
        print(
            f"\n===== SET Run | Sparsity={target_sparsity:.2f} | Optimizer={opt_name} ====="
        )

        model, masks = build_sparse_model(target_sparsity)
        optimizer = build_optimizer(model, opt_name)
        history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

        for epoch in range(epochs):
            model.train()
            running_loss = 0.0
            correct = 0
            total = 0

            for inputs, labels in trainloader:
                inputs, labels = inputs.to(device), labels.to(device)

                optimizer.zero_grad()
                outputs = model(inputs)
                loss = criterion(outputs, labels)
                loss.backward()
                torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
                apply_masks_to_grads(model, masks)
                optimizer.step()
                apply_masks(model, masks)

                running_loss += loss.item()
                _, predicted = outputs.max(1)
                total += labels.size(0)
                correct += predicted.eq(labels).sum().item()

            if (epoch + 1) % rewire_interval == 0:
                prune_and_regrow(model, masks, prune_fraction)
                apply_masks(model, masks)

            total_active = sum(mask.sum().item() for mask in masks.values())
            total_possible = sum(mask.numel() for mask in masks.values())
            current_sparsity = 1 - total_active / total_possible
```

```
            train_loss = running_loss / len(trainloader)
            train_acc = 100.0 * correct / total
            if epoch % 2 == 0:
                test_loss, test_acc = evaluate(model, testloader, criterion)
            else:
                test_loss, test_acc = None, None

            history["train_loss"].append(train_loss)
            history["train_acc"].append(train_acc)
            history["test_loss"].append(test_loss)
            history["test_acc"].append(test_acc)

            print(
                f"Epoch [{epoch + 1}/{epochs}] | "
                f"Train Loss: {train_loss:.4f} | "
                f"Train Acc: {train_acc:.2f}% | "
                f"Test Loss: {test_loss:.4f} | "
                f"Test Acc: {test_acc:.2f}% | "
                f"Current Sparsity: {current_sparsity:.4f}"
            )

        experiment_results = {
            "sparsity": target_sparsity,
            "optimizer": opt_name,
            "method": "SET",
            "prune_fraction": prune_fraction,
            "rewire_interval": rewire_interval,
            "epochs": epochs,
            "train_loss": history["train_loss"],
            "train_acc": history["train_acc"],
            "test_loss": history["test_loss"],
            "test_acc": history["test_acc"],
            "masks": masks,
            "model_state": model.state_dict(),
        }

        results_path = os.path.join(
            PROJECT_DIR,
            "results",
            f"task_2_{opt_name}_{int(target_sparsity * 100)}_sparsity.joblib",
        )
        print("Saving to:", results_path)
```

```
        print("Exists parent folder:", os.path.exists(os.path.dirname(results_path)))
        joblib.dump(experiment_results, results_path)
        print(f"Saved results to {results_path}")

        all_results[opt_name][target_sparsity] = experiment_results

print("\nFinished all sparsity and optimizer runs.")
```

```
===== SET Run | Sparsity=0.90 | Optimizer=sgd =====
Epoch [1/15] | Train Loss: 1.5202 | Train Acc: 43.78% | Test Loss: 30.8960 | Test Acc: 10.00%
Epoch [2/15] | Train Loss: 1.0962 | Train Acc: 60.91% | Test Loss: 191.1856 | Test Acc: 10.0
Epoch [3/15] | Train Loss: 0.9351 | Train Acc: 67.04% | Test Loss: 8.4291 | Test Acc: 19.88%
Epoch [4/15] | Train Loss: 0.8412 | Train Acc: 70.56% | Test Loss: 78.1061 | Test Acc: 10.00%
Epoch [5/15] | Train Loss: 0.7828 | Train Acc: 72.67% | Test Loss: 13.9045 | Test Acc: 10.00%
Epoch [6/15] | Train Loss: 0.7402 | Train Acc: 74.26% | Test Loss: 10.6854 | Test Acc: 10.41%
Epoch [7/15] | Train Loss: 0.7011 | Train Acc: 75.49% | Test Loss: 4.9236 | Test Acc: 22.86%
Epoch [8/15] | Train Loss: 0.6900 | Train Acc: 75.77% | Test Loss: 2.5844 | Test Acc: 31.86%
Epoch [9/15] | Train Loss: 0.6657 | Train Acc: 76.81% | Test Loss: 5.0794 | Test Acc: 20.52%
Epoch [10/15] | Train Loss: 0.6446 | Train Acc: 77.53% | Test Loss: 8.2781 | Test Acc: 11.40%
Epoch [11/15] | Train Loss: 0.6315 | Train Acc: 78.15% | Test Loss: 1.9949 | Test Acc: 41.38%
Epoch [12/15] | Train Loss: 0.6210 | Train Acc: 78.51% | Test Loss: 1.3353 | Test Acc: 55.88%
Epoch [13/15] | Train Loss: 0.6079 | Train Acc: 78.94% | Test Loss: 6.8151 | Test Acc: 12.39%
Epoch [14/15] | Train Loss: 0.6026 | Train Acc: 78.95% | Test Loss: 5.5828 | Test Acc: 17.93%
Epoch [15/15] | Train Loss: 0.5901 | Train Acc: 79.48% | Test Loss: 14.4855 | Test Acc: 10.2
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_sgd_9
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_

===== SET Run | Sparsity=0.95 | Optimizer=sgd =====
Epoch [1/15] | Train Loss: 1.5244 | Train Acc: 43.60% | Test Loss: 48.6423 | Test Acc: 10.00%
Epoch [2/15] | Train Loss: 1.1783 | Train Acc: 58.05% | Test Loss: 1300.7389 | Test Acc: 9.7
Epoch [3/15] | Train Loss: 1.0474 | Train Acc: 62.85% | Test Loss: 789.1155 | Test Acc: 10.0
Epoch [4/15] | Train Loss: 0.9544 | Train Acc: 66.33% | Test Loss: 271.2164 | Test Acc: 11.58
Epoch [5/15] | Train Loss: 0.8981 | Train Acc: 68.47% | Test Loss: 585.2795 | Test Acc: 10.73
Epoch [6/15] | Train Loss: 0.8562 | Train Acc: 70.25% | Test Loss: 41.4276 | Test Acc: 10.10%
Epoch [7/15] | Train Loss: 0.8152 | Train Acc: 71.56% | Test Loss: 2.6659 | Test Acc: 34.19%
Epoch [8/15] | Train Loss: 0.7940 | Train Acc: 72.12% | Test Loss: 4.0874 | Test Acc: 22.20%
Epoch [9/15] | Train Loss: 0.7721 | Train Acc: 72.98% | Test Loss: 4.8606 | Test Acc: 18.50%
Epoch [10/15] | Train Loss: 0.7478 | Train Acc: 73.89% | Test Loss: 4.7573 | Test Acc: 25.46%
Epoch [11/15] | Train Loss: 0.7351 | Train Acc: 74.32% | Test Loss: 5.8130 | Test Acc: 24.19%
Epoch [12/15] | Train Loss: 0.7171 | Train Acc: 74.93% | Test Loss: 4.7112 | Test Acc: 19.94%
```

```
Epoch [13/15] | Train Loss: 0.7039 | Train Acc: 75.43% | Test Loss: 2.7682 | Test Acc: 29.05%
Epoch [14/15] | Train Loss: 0.6892 | Train Acc: 75.88% | Test Loss: 1.6474 | Test Acc: 45.75%
Epoch [15/15] | Train Loss: 0.6771 | Train Acc: 76.45% | Test Loss: 1.4622 | Test Acc: 52.52%
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_sgd_9
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2

===== SET Run | Sparsity=0.98 | Optimizer=sgd =====
Epoch [1/15] | Train Loss: 1.7405 | Train Acc: 35.96% | Test Loss: 110.1819 | Test Acc: 10.36
Epoch [2/15] | Train Loss: 1.4534 | Train Acc: 47.74% | Test Loss: 317.7496 | Test Acc: 11.00
Epoch [3/15] | Train Loss: 1.3136 | Train Acc: 52.89% | Test Loss: 624.8915 | Test Acc: 10.01
Epoch [4/15] | Train Loss: 1.2377 | Train Acc: 55.96% | Test Loss: 33.4417 | Test Acc: 7.28%
Epoch [5/15] | Train Loss: 1.1774 | Train Acc: 58.73% | Test Loss: 6.8860 | Test Acc: 13.17%
Epoch [6/15] | Train Loss: 1.1208 | Train Acc: 60.66% | Test Loss: 21.6749 | Test Acc: 11.04%
Epoch [7/15] | Train Loss: 1.1001 | Train Acc: 61.23% | Test Loss: 46.4277 | Test Acc: 10.02%
Epoch [8/15] | Train Loss: 1.0729 | Train Acc: 62.08% | Test Loss: 3.1938 | Test Acc: 26.75%
Epoch [9/15] | Train Loss: 1.0461 | Train Acc: 63.30% | Test Loss: 2.5768 | Test Acc: 27.65%
Epoch [10/15] | Train Loss: 1.0243 | Train Acc: 63.95% | Test Loss: 5.2703 | Test Acc: 17.90%
Epoch [11/15] | Train Loss: 1.0100 | Train Acc: 64.44% | Test Loss: 2.2389 | Test Acc: 26.69%
Epoch [12/15] | Train Loss: 0.9881 | Train Acc: 65.47% | Test Loss: 6.5801 | Test Acc: 16.52%
Epoch [13/15] | Train Loss: 0.9746 | Train Acc: 65.68% | Test Loss: 1.3289 | Test Acc: 53.78%
Epoch [14/15] | Train Loss: 0.9658 | Train Acc: 66.12% | Test Loss: 17.0363 | Test Acc: 14.60
Epoch [15/15] | Train Loss: 0.9491 | Train Acc: 66.63% | Test Loss: 22.9139 | Test Acc: 10.63
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_sgd_9
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2

===== SET Run | Sparsity=0.90 | Optimizer=adam =====
Epoch [1/15] | Train Loss: 1.6281 | Train Acc: 40.38% | Test Loss: 11.7294 | Test Acc: 9.78%
Epoch [2/15] | Train Loss: 1.2044 | Train Acc: 56.90% | Test Loss: 2.9840 | Test Acc: 22.81%
Epoch [3/15] | Train Loss: 1.0463 | Train Acc: 62.52% | Test Loss: 4.1565 | Test Acc: 16.73%
Epoch [4/15] | Train Loss: 0.9584 | Train Acc: 65.96% | Test Loss: 1.8063 | Test Acc: 40.51%
Epoch [5/15] | Train Loss: 0.8885 | Train Acc: 68.47% | Test Loss: 10.7280 | Test Acc: 8.16%
Epoch [6/15] | Train Loss: 0.8418 | Train Acc: 70.41% | Test Loss: 11.4216 | Test Acc: 12.71%
Epoch [7/15] | Train Loss: 0.7958 | Train Acc: 71.94% | Test Loss: 8.5269 | Test Acc: 13.40%
Epoch [8/15] | Train Loss: 0.7536 | Train Acc: 73.70% | Test Loss: 8.6733 | Test Acc: 17.26%
Epoch [9/15] | Train Loss: 0.7180 | Train Acc: 74.78% | Test Loss: 21.7479 | Test Acc: 10.00%
Epoch [10/15] | Train Loss: 0.6901 | Train Acc: 75.84% | Test Loss: 29.4283 | Test Acc: 15.76
Epoch [11/15] | Train Loss: 0.6635 | Train Acc: 76.72% | Test Loss: 2.4364 | Test Acc: 36.55%
Epoch [12/15] | Train Loss: 0.6501 | Train Acc: 77.26% | Test Loss: 4.0913 | Test Acc: 25.84%
Epoch [13/15] | Train Loss: 0.6239 | Train Acc: 78.18% | Test Loss: 3.2540 | Test Acc: 28.78%
Epoch [14/15] | Train Loss: 0.6083 | Train Acc: 78.65% | Test Loss: 2.5789 | Test Acc: 30.84%
Epoch [15/15] | Train Loss: 0.5932 | Train Acc: 79.12% | Test Loss: 7.3574 | Test Acc: 11.05%
```

```
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_adam_
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2

===== SET Run | Sparsity=0.95 | Optimizer=adam =====
Epoch [1/15] | Train Loss: 1.7889 | Train Acc: 34.03% | Test Loss: 3.9473 | Test Acc: 10.49%
Epoch [2/15] | Train Loss: 1.4160 | Train Acc: 48.72% | Test Loss: 3.2101 | Test Acc: 15.15%
Epoch [3/15] | Train Loss: 1.2394 | Train Acc: 55.91% | Test Loss: 2.7019 | Test Acc: 24.18%
Epoch [4/15] | Train Loss: 1.1374 | Train Acc: 59.70% | Test Loss: 20.8476 | Test Acc: 10.00%
Epoch [5/15] | Train Loss: 1.0538 | Train Acc: 62.59% | Test Loss: 5.1133 | Test Acc: 11.39%
Epoch [6/15] | Train Loss: 0.9884 | Train Acc: 64.75% | Test Loss: 11.0938 | Test Acc: 10.97%
Epoch [7/15] | Train Loss: 0.9469 | Train Acc: 66.49% | Test Loss: 1.7519 | Test Acc: 36.96%
Epoch [8/15] | Train Loss: 0.9019 | Train Acc: 68.03% | Test Loss: 9.7138 | Test Acc: 12.81%
Epoch [9/15] | Train Loss: 0.8641 | Train Acc: 69.36% | Test Loss: 4.9755 | Test Acc: 17.63%
Epoch [10/15] | Train Loss: 0.8357 | Train Acc: 70.46% | Test Loss: 35.1490 | Test Acc: 10.0%
Epoch [11/15] | Train Loss: 0.8022 | Train Acc: 71.69% | Test Loss: 1.7824 | Test Acc: 45.93%
Epoch [12/15] | Train Loss: 0.7804 | Train Acc: 72.61% | Test Loss: 3.8463 | Test Acc: 22.34%
Epoch [13/15] | Train Loss: 0.7644 | Train Acc: 73.14% | Test Loss: 2.8786 | Test Acc: 28.95%
Epoch [14/15] | Train Loss: 0.7389 | Train Acc: 74.00% | Test Loss: 2.5066 | Test Acc: 31.18%
Epoch [15/15] | Train Loss: 0.7223 | Train Acc: 74.59% | Test Loss: 2.2989 | Test Acc: 34.61%
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_adam_
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2

===== SET Run | Sparsity=0.98 | Optimizer=adam =====
Epoch [1/15] | Train Loss: 2.0154 | Train Acc: 25.87% | Test Loss: 3.8615 | Test Acc: 13.57%
Epoch [2/15] | Train Loss: 1.8265 | Train Acc: 35.35% | Test Loss: 2.1751 | Test Acc: 25.17%
Epoch [3/15] | Train Loss: 1.6782 | Train Acc: 40.46% | Test Loss: 8.2002 | Test Acc: 14.34%
Epoch [4/15] | Train Loss: 1.6150 | Train Acc: 42.34% | Test Loss: 2.8219 | Test Acc: 16.49%
Epoch [5/15] | Train Loss: 1.6700 | Train Acc: 37.94% | Test Loss: 2.1712 | Test Acc: 23.04%
Epoch [6/15] | Train Loss: 1.5576 | Train Acc: 44.96% | Test Loss: 5.8019 | Test Acc: 10.53%
Epoch [7/15] | Train Loss: 1.4928 | Train Acc: 45.91% | Test Loss: 2.2870 | Test Acc: 25.15%
Epoch [8/15] | Train Loss: 1.4238 | Train Acc: 48.08% | Test Loss: 2.1149 | Test Acc: 26.20%
Epoch [9/15] | Train Loss: 1.3557 | Train Acc: 50.49% | Test Loss: 2.8180 | Test Acc: 22.18%
Epoch [10/15] | Train Loss: 1.2940 | Train Acc: 52.91% | Test Loss: 2.8062 | Test Acc: 29.90%
Epoch [11/15] | Train Loss: 1.2108 | Train Acc: 57.63% | Test Loss: 3.2049 | Test Acc: 22.74%
Epoch [12/15] | Train Loss: 1.1709 | Train Acc: 58.89% | Test Loss: 2.5090 | Test Acc: 20.79%
Epoch [13/15] | Train Loss: 1.1919 | Train Acc: 58.64% | Test Loss: 2.0194 | Test Acc: 32.08%
Epoch [14/15] | Train Loss: 1.1246 | Train Acc: 60.52% | Test Loss: 8.5399 | Test Acc: 12.55%
Epoch [15/15] | Train Loss: 1.1059 | Train Acc: 61.29% | Test Loss: 2.7871 | Test Acc: 20.33%
Saving to: /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2_adam_
Exists parent folder: True
Saved results to /content/drive/MyDrive/MIDS/Spring26/ECE685D_DL/final_project/results/task_2
```

```
Finished all sparsity and optimizer runs.
```

3. Analysis and Comparison

```
import os
import sys
from pathlib import Path

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


def infer_project_dir(start_path=None):
    """Find project root by looking for models.py and results/ folder."""
    start = Path(start_path or os.getcwd()).resolve()
    for candidate in [start, *start.parents]:
        if (candidate / "models.py").exists() and (candidate / "results").exists():
            return str(candidate)
    return str(start)


PROJECT_DIR = infer_project_dir()
if PROJECT_DIR not in sys.path:
    sys.path.append(PROJECT_DIR)

sns.set_theme(style="whitegrid", context="notebook", palette="deep")
os.makedirs(os.path.join(PROJECT_DIR, "img"), exist_ok=True)

print(f"Using local PROJECT_DIR: {PROJECT_DIR}")
```

```
Using local PROJECT_DIR: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project
```

```
import torch
import joblib

# Patch torch.load to force CPU when loading serialized tensors from joblib.
_original_torch_load = torch.load
```

```
def cpu_torch_load(*args, **kwargs):
    kwargs["map_location"] = torch.device("cpu")
    return _original_torch_load(*args, **kwargs)


torch.load = cpu_torch_load


def load_task_results(task_id, optimizers, sparsity_levels, output_dir):
    results = {opt: {} for opt in optimizers}
    for opt in optimizers:
        for s in sparsity_levels:
            file_path = os.path.join(
                output_dir,
                f"task_{task_id}_{opt}_{int(s * 100)}_sparsity.joblib",
            )
            if os.path.exists(file_path):
                results[opt][s] = joblib.load(file_path)
                print(f"Loaded: {file_path}")
            else:
                print(f"Missing: {file_path}")
    return results


analysis_dir = os.path.join(PROJECT_DIR, "results")
if not os.path.exists(analysis_dir):
    raise FileNotFoundError(
        f"Could not find local results directory at: {analysis_dir}. "
        "Set PROJECT_DIR to your project root if needed."
    )

analysis_sparsities = list(globals().get("sparsity_levels", [0.90, 0.95, 0.98]))
analysis_opts = list(globals().get("optimizers", ["sgd", "adam"]))

results_task_1 = load_task_results(1, analysis_opts, analysis_sparsities, analysis_dir)
results_task_2 = load_task_results(2, analysis_opts, analysis_sparsities, analysis_dir)
```

```
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_sgd_90_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_sgd_95_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_sgd_98_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_adam_90_sparsi
```

```
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_adam_95_sparsi
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_1_adam_98_sparsi
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_sgd_90_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_sgd_95_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_sgd_98_sparsit
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_adam_90_sparsi
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_adam_95_sparsi
Loaded: /Users/ammylin/Desktop/ECE685D_DL/ECE685D_Final_Project/results/task_2_adam_98_sparsi
```

```python
def plot_training_and_validation(results, task_label="Task 2 (SET)"):
    for s in analysis_sparsities:
        rows = []

        for opt in analysis_opts:
            if s in results[opt]:
                n_epochs = len(results[opt][s]["train_loss"])
                for epoch in range(n_epochs):
                    rows.append(
                        {
                            "epoch": epoch + 1,
                            "optimizer": opt.upper(),
                            "split": "Train",
                            "loss": results[opt][s]["train_loss"][epoch],
                            "accuracy": results[opt][s]["train_acc"][epoch],
                        }
                    )
                    rows.append(
                        {
                            "epoch": epoch + 1,
                            "optimizer": opt.upper(),
                            "split": "Test",
                            "loss": results[opt][s]["test_loss"][epoch],
                            "accuracy": results[opt][s]["test_acc"][epoch],
                        }
                    )

        if not rows:
            print(f"No runs available for sparsity={s:.2f}")
            continue

        df = pd.DataFrame(rows)
```

```
        fig, axes = plt.subplots(2, 2, figsize=(14, 9), sharex=True)
        fig.suptitle(
            f"{task_label}: Training Dynamics at {int(s * 100)}% Sparsity",
            fontsize=16,
            weight="bold",
        )

        sns.lineplot(
            data=df[df["split"] == "Train"],
            x="epoch",
            y="loss",
            hue="optimizer",
            linewidth=2.2,
            ax=axes[0, 0],
        )
        axes[0, 0].set_title("Train Loss")
        axes[0, 0].set_ylabel("Loss")
        axes[0, 0].legend(title="")

        sns.lineplot(
            data=df[df["split"] == "Test"],
            x="epoch",
            y="loss",
            hue="optimizer",
            linewidth=2.2,
            ax=axes[0, 1],
        )
        axes[0, 1].set_title("Test Loss")
        axes[0, 1].set_ylabel("Loss")
        axes[0, 1].legend(title="")

        sns.lineplot(
            data=df[df["split"] == "Train"],
            x="epoch",
            y="accuracy",
            hue="optimizer",
            linewidth=2.2,
            ax=axes[1, 0],
        )
        axes[1, 0].set_title("Train Accuracy")
        axes[1, 0].set_xlabel("Epoch")
        axes[1, 0].set_ylabel("Accuracy (%)")
```

```
        axes[1, 0].legend(title="")

        sns.lineplot(
            data=df[df["split"] == "Test"],
            x="epoch",
            y="accuracy",
            hue="optimizer",
            linewidth=2.2,
            ax=axes[1, 1],
        )
        axes[1, 1].set_title("Test Accuracy")
        axes[1, 1].set_xlabel("Epoch")
        axes[1, 1].set_ylabel("Accuracy (%)")
        axes[1, 1].legend(title="")

        for ax in axes.flat:
            ax.grid(True, alpha=0.3)

        plt.tight_layout()
        plt.savefig(
            os.path.join(PROJECT_DIR, "img", f"task_2_curves_{int(s * 100)}.png"),
            dpi=300,
            bbox_inches="tight",
        )
        plt.show()
```

```
def plot_accuracy_vs_sparsity(results, task_label="Task 2 (SET)"):
    rows = []
    for opt in analysis_opts:
        for s in analysis_sparsities:
            if s in results[opt]:
                rows.append(
                    {
                        "optimizer": opt.upper(),
                        "sparsity": int(s * 100),
                        "final_test_acc": results[opt][s]["test_acc"][-1],
                    }
                )

    if not rows:
        print("No data available for accuracy-vs-sparsity plot.")
        return None
```

```
    df = pd.DataFrame(rows)

    plt.figure(figsize=(6, 4))
    ax = sns.lineplot(
        data=df,
        x="sparsity",
        y="final_test_acc",
        hue="optimizer",
        marker="o",
        linewidth=2.4,
    )

    for _, row in df.iterrows():
        ax.text(
            row["sparsity"],
            row["final_test_acc"],
            f"{row['final_test_acc']:.1f}",
            fontsize=8,
            ha="center",
            va="bottom",
        )

    ax.set_title(f"{task_label}: Final Test Accuracy vs Sparsity", weight="bold")
    ax.set_xlabel("Sparsity (%)")
    ax.set_ylabel("Final Test Accuracy (%)")
    ax.set_xticks([90, 95, 98])
    ax.legend(title="")
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(
        os.path.join(PROJECT_DIR, "img", "task_2_accuracy_vs_sparsity.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show()

    return df.sort_values(["optimizer", "sparsity"]).reset_index(drop=True)
```

```
def compare_task1_vs_task2(results_task_1, results_task_2):
    rows = []
```

```
    for opt in analysis_opts:
        for s in analysis_sparsities:
            if s in results_task_1[opt] and s in results_task_2[opt]:
                baseline_acc = results_task_1[opt][s]["test_acc"][-1]
                set_acc = results_task_2[opt][s]["test_acc"][-1]
                rows.append(
                    {
                        "optimizer": opt.upper(),
                        "sparsity": int(s * 100),
                        "task_1_static": baseline_acc,
                        "task_2_set": set_acc,
                        "delta_set_minus_static": set_acc - baseline_acc,
                    }
                )

    if not rows:
        print("No overlapping Task 1/Task 2 runs found.")
        return None

    df = (
        pd.DataFrame(rows).sort_values(["optimizer", "sparsity"]).reset_index(drop=True)
    )

    long_df = df.melt(
        id_vars=["optimizer", "sparsity"],
        value_vars=["task_1_static", "task_2_set"],
        var_name="method",
        value_name="final_test_acc",
    )
    long_df["method"] = long_df["method"].map(
        {"task_1_static": "Task 1 Static", "task_2_set": "Task 2 SET"}
    )

    plt.figure(figsize=(8, 4.5))
    ax = sns.barplot(
        data=long_df,
        x="sparsity",
        y="final_test_acc",
        hue="method",
        errorbar=None,
    )
    ax.set_title("Task 1 vs Task 2: Final Test Accuracy", weight="bold")
```

```
    ax.set_xlabel("Sparsity (%)")
    ax.set_ylabel("Final Test Accuracy (%)")
    ax.grid(True, axis="y", alpha=0.25)
    ax.legend(title="")
    plt.tight_layout()
    plt.savefig(
        os.path.join(PROJECT_DIR, "img", "task_1_vs_task_2_accuracy.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show()

    delta_pivot = df.pivot(
        index="sparsity", columns="optimizer", values="delta_set_minus_static"
    )
    plt.figure(figsize=(6, 3.8))
    sns.heatmap(
        delta_pivot,
        annot=True,
        fmt=".2f",
        cmap="RdYlGn",
        center=0,
        cbar_kws={"label": "SET - Static Accuracy (pp)"},
    )
    plt.title("SET Gain/Loss Over Static Sparse Baseline", weight="bold")
    plt.xlabel("Optimizer")
    plt.ylabel("Sparsity (%)")
    plt.tight_layout()
    plt.savefig(
        os.path.join(PROJECT_DIR, "img", "task_2_delta_heatmap.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show()

    return df
```

Layer- and Component-Level Sparsity

This block computes sparsity statistics directly from the saved binary masks.

What it does: - Classifies each layer as `conv`, `fc`, or `other` from the layer name. - Computes

per-layer sparsity using sparsity $= 1 - \frac{\|W\|_0}{|W|}$ where $\|W\|_0$ is the number of active (nonzero) mask entries. - Aggregates to component-level sparsity (conv/fc/other) for easier comparison.

How to read the plot: - Higher bar means more sparse (fewer active connections). - Compare Task 1 vs Task 2 at the same optimizer/sparsity to see where SET changes connectivity distribution.

```
def layer_type_from_name(name):
    lname = name.lower()
    if "conv" in lname:
        return "conv"
    if "fc" in lname or "linear" in lname:
        return "fc"
    return "other"


def build_layer_sparsity_table(results, method_label):
    rows = []
    for opt in analysis_opts:
        for s in analysis_sparsities:
            run = results.get(opt, {}).get(s)
            if run is None:
                continue

            masks = run.get("masks", {})
            for layer_name, mask in masks.items():
                total = int(mask.numel())
                active = int(torch.count_nonzero(mask).item())
                sparsity = 1.0 - (active / total)
                rows.append(
                    {
                        "method": method_label,
                        "optimizer": opt.upper(),
                        "sparsity_target": int(s * 100),
                        "layer": layer_name,
                        "layer_type": layer_type_from_name(layer_name),
                        "active_weights": active,
                        "total_weights": total,
                        "actual_sparsity": sparsity,
                    }
                )

    if not rows:
```

```
        return pd.DataFrame()

    return pd.DataFrame(rows)


def component_summary(layer_df):
    if layer_df.empty:
        return pd.DataFrame()

    grouped = layer_df.groupby(
        ["method", "optimizer", "sparsity_target", "layer_type"], as_index=False
    ).agg(
        active_weights=("active_weights", "sum"),
        total_weights=("total_weights", "sum"),
    )
    grouped["actual_sparsity"] = 1.0 - (
        grouped["active_weights"] / grouped["total_weights"]
    )
    return grouped.sort_values(["method", "optimizer", "sparsity_target", "layer_type"])


def plot_component_sparsity(comp_df):
    if comp_df.empty:
        print("No component-level sparsity data available.")
        return

    plot_df = comp_df.copy()
    plot_df["actual_sparsity_pct"] = plot_df["actual_sparsity"] * 100
    plot_df["config"] = (
        plot_df["method"]
        + " | "
        + plot_df["optimizer"]
        + " | "
        + plot_df["sparsity_target"].astype(str)
        + "%"
    )

    plt.figure(figsize=(12, 5))
    ax = sns.barplot(
        data=plot_df,
        x="config",
        y="actual_sparsity_pct",
```

```
        hue="layer_type",
    )
    ax.set_title("Component-Wise Sparsity (Conv vs FC vs Other)", weight="bold")
    ax.set_xlabel("Method | Optimizer | Target Sparsity")
    ax.set_ylabel("Actual Sparsity (%)")
    ax.tick_params(axis="x", rotation=35)
    ax.grid(True, axis="y", alpha=0.25)
    ax.legend(title="Layer Type")
    plt.tight_layout()
    plt.savefig(
        os.path.join(PROJECT_DIR, "img", "task_2_component_sparsity.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show()
```

Connectivity Similarity

This block quantifies how similar connectivity patterns are across methods.

Metric used: - **Mask Jaccard similarity** between two masks: $J(A,B) = \frac{|A \cap B|}{|A \cup B|}$ - $J = 1$ means identical active connections; lower values mean larger topology differences.

What is reported: - **Task 1 vs Task 2 overlap** at the end of training (heatmap by optimizer and sparsity).

Interpretation: - Lower Task1-vs-Task2 Jaccard suggests SET found a different connectivity structure than static sparse training.

```
def mask_jaccard(mask_a, mask_b):
    a = mask_a.bool()
    b = mask_b.bool()
    inter = torch.logical_and(a, b).sum().item()
    union = torch.logical_or(a, b).sum().item()
    if union == 0:
        return 1.0
    return inter / union


def connectivity_overlap_task1_vs_task2(results_task_1, results_task_2):
    rows = []
    for opt in analysis_opts:
        for s in analysis_sparsities:
```

```
            run1 = results_task_1.get(opt, {}).get(s)
            run2 = results_task_2.get(opt, {}).get(s)
            if run1 is None or run2 is None:
                continue

            masks1 = run1.get("masks", {})
            masks2 = run2.get("masks", {})
            common_layers = sorted(set(masks1.keys()).intersection(masks2.keys()))
            if not common_layers:
                continue

            layer_scores = []
            for layer in common_layers:
                score = mask_jaccard(masks1[layer], masks2[layer])
                layer_scores.append(score)
                rows.append(
                    {
                        "optimizer": opt.upper(),
                        "sparsity_target": int(s * 100),
                        "layer": layer,
                        "mask_jaccard": score,
                    }
                )

            rows.append(
                {
                    "optimizer": opt.upper(),
                    "sparsity_target": int(s * 100),
                    "layer": "__OVERALL__",
                    "mask_jaccard": float(sum(layer_scores) / len(layer_scores)),
                }
            )

    if not rows:
        return pd.DataFrame(), pd.DataFrame()

    overlap_df = pd.DataFrame(rows)
    overall_df = overlap_df[overlap_df["layer"] == "__OVERALL__"].copy()
    return overlap_df, overall_df


def plot_connectivity_overlap(overall_df):
```

```
    if overall_df.empty:
        print("No overlap data available for Task 1 vs Task 2 connectivity.")
        return

    pivot = overall_df.pivot(
        index="sparsity_target", columns="optimizer", values="mask_jaccard"
    )

    plt.figure(figsize=(6, 4))
    sns.heatmap(
        pivot,
        annot=True,
        fmt=".3f",
        cmap="Blues",
        vmin=0,
        vmax=1,
        cbar_kws={"label": "Mask Jaccard Similarity"},
    )
    plt.title("Connectivity Overlap: Task 1 Static vs Task 2 SET", weight="bold")
    plt.xlabel("Optimizer")
    plt.ylabel("Target Sparsity (%)")
    plt.tight_layout()
    plt.savefig(
        os.path.join(PROJECT_DIR, "img", "task_2_connectivity_overlap_heatmap.png"),
        dpi=300,
        bbox_inches="tight",
    )
    plt.show()
```

Run All Analysis and Summaries

This final cell executes the full analysis pipeline and prints compact tables for reporting.

Outputs generated: - Training/validation dynamics (Task 2) - Accuracy vs sparsity (Task 2) - Task 1 vs Task 2 final-accuracy bar comparison - SET minus Static delta heatmap - Component-level sparsity summary - Connectivity overlap summary

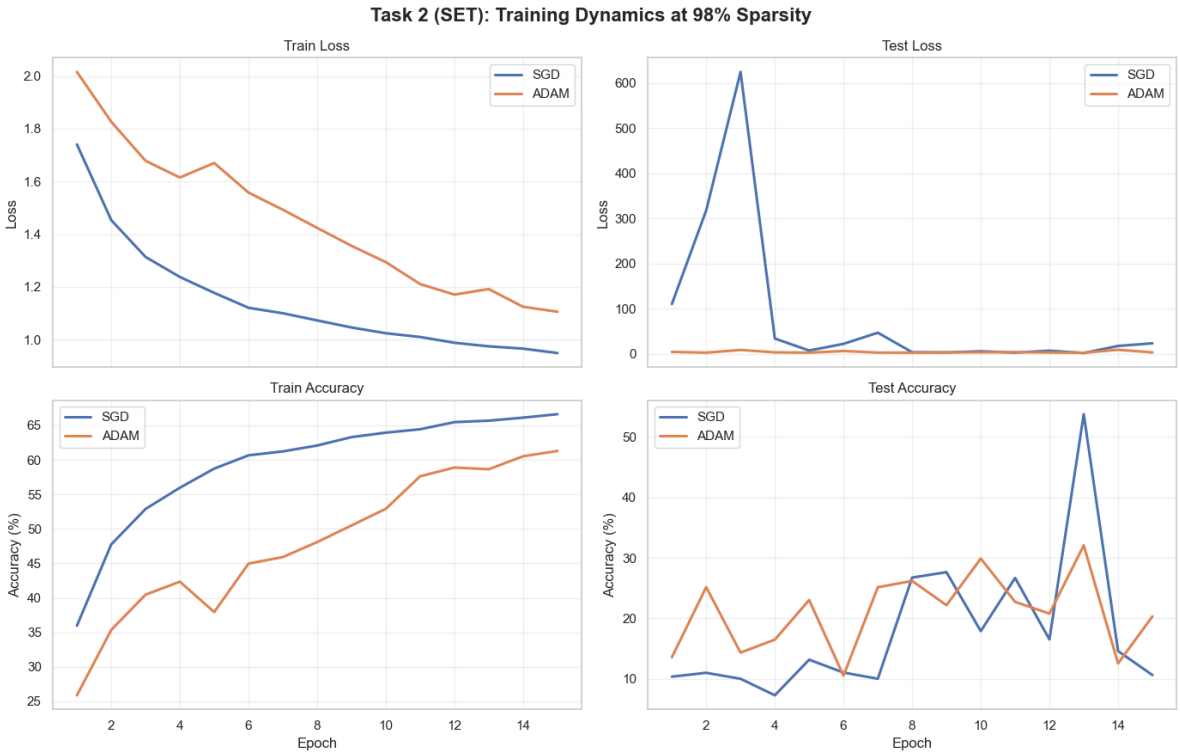
Saved figures are written to the `img/` folder.

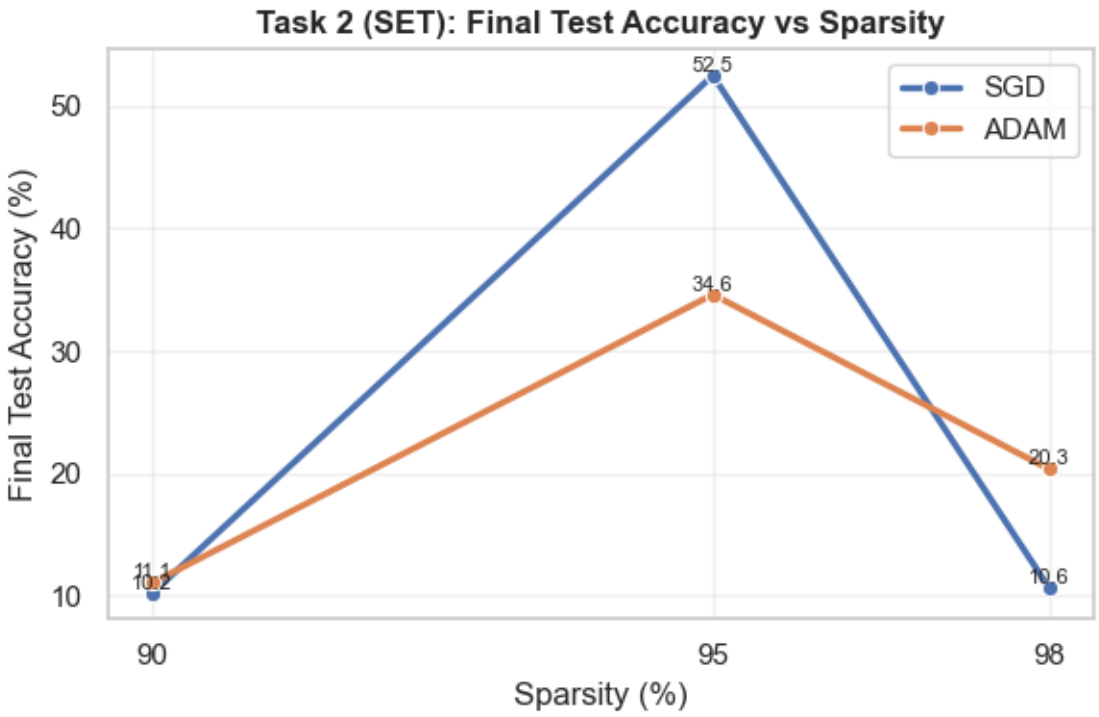
```
plot_training_and_validation(results_task_2, task_label="Task 2 (SET)")

task2_summary = plot_accuracy_vs_sparsity(results_task_2, task_label="Task 2 (SET)")
```

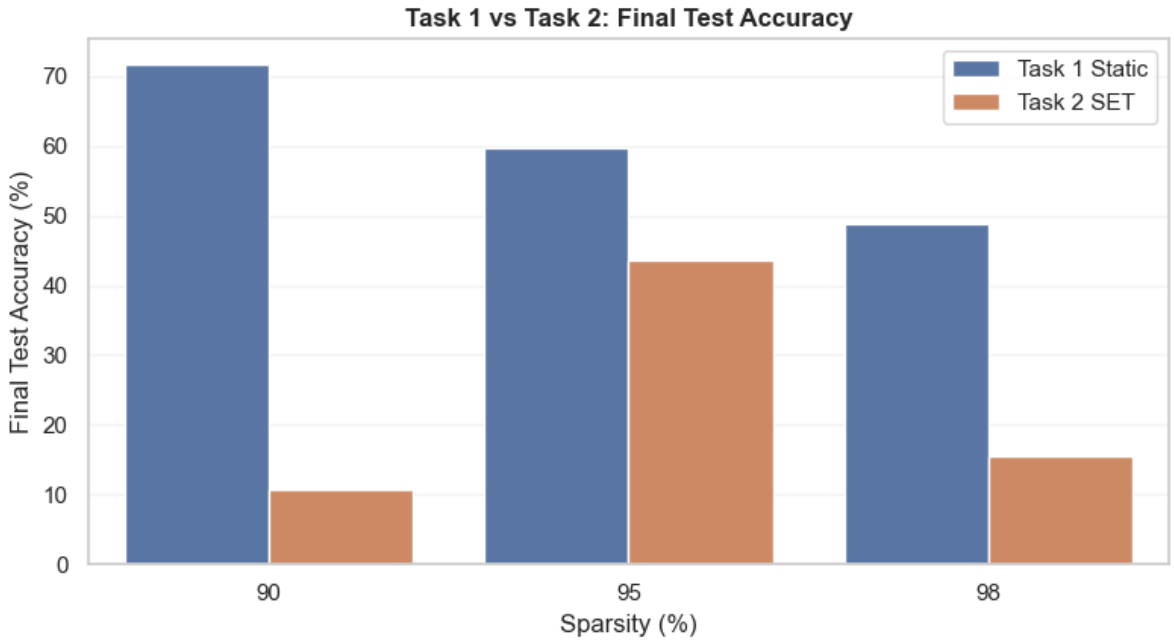
```
comparison_df = compare_task1_vs_task2(results_task_1, results_task_2)

layer_df_task1 = build_layer_sparsity_table(
    results_task_1, method_label="Task 1 Static"
)
layer_df_task2 = build_layer_sparsity_table(results_task_2, method_label="Task 2 SET")
layer_df_all = pd.concat([layer_df_task1, layer_df_task2], ignore_index=True)
component_df = component_summary(layer_df_all)
plot_component_sparsity(component_df)

_, connectivity_overall_df = connectivity_overlap_task1_vs_task2(
    results_task_1, results_task_2
)
plot_connectivity_overlap(connectivity_overall_df)

if task2_summary is not None:
    print("\nTask 2 final accuracy summary:")
    display(task2_summary)

if comparison_df is not None:
    print("\nTask 1 vs Task 2 comparison (positive delta means SET is better):")
    display(comparison_df)
    print("\nAverage delta by optimizer:")
    display(
        comparison_df.groupby("optimizer", as_index=False)[
            "delta_set_minus_static"
        ].mean()
    )

if not component_df.empty:
    print("\nComponent-wise sparsity summary:")
    display(component_df)

if not connectivity_overall_df.empty:
    print("\nConnectivity overlap summary (Task 1 vs Task 2):")
    display(connectivity_overall_df)
```

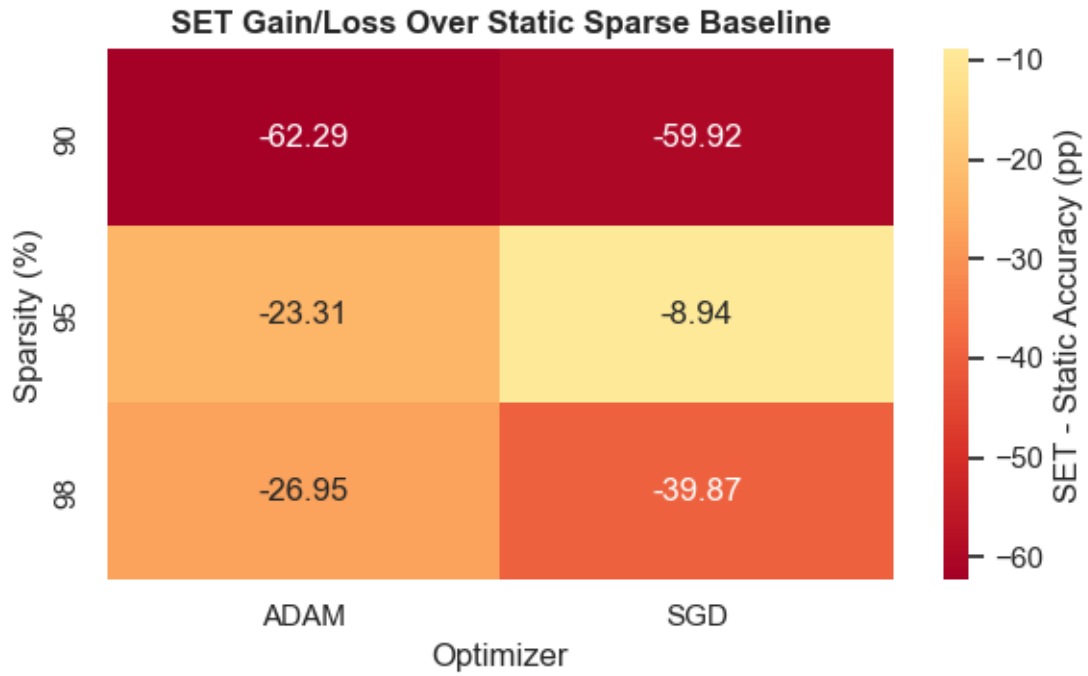

Task 2 (SET): Training Dynamics at 90% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch


Task 2 (SET): Training Dynamics at 95% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch


Task 2 (SET): Training Dynamics at 98% Sparsity
Train Loss
Test Loss
Train Accuracy
Test Accuracy
SGD
ADAM
Loss
Accuracy (%)
Epoch


Task 2 (SET): Final Test Accuracy vs Sparsity
SGD
ADAM
52.5
34.6
20.3
11.1
10.2
10.6
Final Test Accuracy (%)
Sparsity (%)
90
95
98






```
Task 2 final accuracy summary:
```

	optimizer	sparsity	final_test_acc
0	ADAM	90	11.05
1	ADAM	95	34.61
2	ADAM	98	20.33
3	SGD	90	10.22
4	SGD	95	52.52
5	SGD	98	10.63

```
Task 1 vs Task 2 comparison (positive delta means SET is better):
```

	optimizer	sparsity	task_1_static	task_2_set	delta_set_minus_static
0	ADAM	90	73.34	11.05	-62.29
1	ADAM	95	57.92	34.61	-23.31
2	ADAM	98	47.28	20.33	-26.95
3	SGD	90	70.14	10.22	-59.92
4	SGD	95	61.46	52.52	-8.94
5	SGD	98	50.50	10.63	-39.87

```
Average delta by optimizer:
```

	optimizer	delta_set_minus_static
0	ADAM	-37.516667
1	SGD	-36.243333